

Nama : Carlos Fadellilah Rama

Kelas : TI-2E

Matkul : PBO 4

1. Implementasi Kelas & Kode (Java)

Di sini saya membuat 3 kelas: `Mesin`, `SepedaMotor`, dan satu kelas tambahan `Mekanik` untuk mencontohkan relasi **Uses-A**.

```
```java
// Kelas 1: Mesin
class Mesin {
 private int kapasitas; // dalam cc
 private String tipeBahanBakar;

 // Constructor
 public Mesin(int kapasitas, String tipeBahanBakar) {
 setKapasitas(kapasitas); // Menggunakan setter untuk validasi awal
 this.tipeBahanBakar = tipeBahanBakar;
 }

 // Getter & Setter dengan Defensive Programming
 public int getKapasitas() {
 return kapasitas;
 }

 public void setKapasitas(int kapasitas) {
 // Defensive Programming 1: Validasi input tidak masuk akal
 if (kapasitas <= 0) {
```

```

 throw new IllegalArgumentException("Kapasitas mesin harus lebih dari 0 cc!");
 }

 this.kapasitas = kapasitas;
}

public String getTipeBahanBakar() {
 return tipeBahanBakar;
}

public void setTipeBahanBakar(String tipeBahanBakar) {
 this.tipeBahanBakar = tipeBahanBakar;
}
}

```

// Kelas 2: SepedaMotor

```

class SepedaMotor {
 private String merek;
 private String warna;
 private int maxSpeed;
 private int kecepatanSaatIni; // Atribut tambahan untuk behavior kecepatan
 private Mesin mesin; // Relasi Has-A

 // Constructor
 public SepedaMotor(String merek, String warna, int maxSpeed, Mesin mesin) {
 this.merek = merek;
 this.warna = warna;
 setMaxSpeed(maxSpeed);
 setMesin(mesin);
 this.kecepatanSaatIni = 0; // Kecepatan awal selalu 0
 }
}

```

```
}
```

```
// Behavior: Menambah Kecepatan
```

```
public void tambahKecepatan(int tambahan) {
```

```
 // Defensive Programming 2: Mencegah nilai negatif dan over-speed
```

```
 if (tambahan < 0) {
```

```
 System.out.println("Gagal: Input penambahan kecepatan tidak boleh negatif!");
```

```
 return;
```

```
 }
```

```
 if (this.kecepatanSaatIni + tambahan > maxSpeed) {
```

```
 this.kecepatanSaatIni = maxSpeed;
```

```
 System.out.println("Peringatan: Kecepatan maksimal (" + maxSpeed + " km/jam) telah tercapai!");
```

```
 } else {
```

```
 this.kecepatanSaatIni += tambahan;
```

```
 System.out.println("Kecepatan motor " + merek + " naik menjadi " + kecepatanSaatIni + " km/jam.");
```

```
 }
```

```
}
```

```
// Behavior: Mengurangi Kecepatan
```

```
public void kurangiKecepatan(int pengurangan) {
```

```
 // Defensive Programming 3: Mencegah kecepatan menjadi minus
```

```
 if (pengurangan < 0) {
```

```
 System.out.println("Gagal: Input pengurangan kecepatan tidak boleh negatif!");
```

```
 return;
```

```
 }
```

```
 if (this.kecepatanSaatIni - pengurangan < 0) {
```

```

 this.kecepatanSaatIni = 0;

 System.out.println("Motor berhenti.");
 } else {

 this.kecepatanSaatIni -= pengurangan;

 System.out.println("Kecepatan motor turun menjadi " + kecepatanSaatIni + "
km/jam.");
 }
}

```

// Getter & Setter

```

public Mesin getMesin() {
 return mesin;
}

```

```

public void setMesin(Mesin mesin) {
 // Defensive Programming 4: Mencegah Null Pointer Exception
 if (mesin == null) {
 throw new IllegalArgumentException("Sepeda motor harus memiliki mesin (tidak
boleh null)!");
 }

 this.mesin = mesin;
}

```

```

public void setMaxSpeed(int maxSpeed) {
 if (maxSpeed <= 0) {
 throw new IllegalArgumentException("Kecepatan maksimal harus lebih dari 0!");
 }

 this.maxSpeed = maxSpeed;
}

```

```

 // ... (Sisa getter/setter untuk merek dan warna bisa ditambahkan serupa)
}

// Kelas 3: Mekanik (Untuk contoh relasi Uses-A)
class Mekanik {
 private String namaMekanik;

 public Mekanik(String nama) {
 this.namaMekanik = nama;
 }

 // Relasi Uses-A: Mekanik menerima objek SepedaMotor sebagai parameter,
 // tapi tidak menyimpannya sebagai atribut.
 public void servisMotor(SepedaMotor motor) {
 System.out.println(namaMekanik + " sedang menservis motor " +
 motor.getMesin().getKapasitas() + "cc.");
 }
}

'''

```

## ### 2. Penerapan Defensive Programming

Pada kode di atas, saya menerapkan beberapa pola **\*\*Defensive Programming\*\*** untuk mencegah error atau kondisi yang tidak masuk akal (bug):

1. **\*\*Mencegah Nilai Ilegal pada Setter:\*\*** Di metode ``setKapasitas`` dan ``setMaxSpeed``, kode akan menolak (melempar `*Exception*`) jika angka yang dimasukkan adalah 0 atau negatif. (Masa iya ada mesin kapasitasnya -100 cc?).
2. **\*\*Mencegah Null Reference (Null Check):\*\*** Di ``setMesin``, kode memastikan objek mesin tidak boleh kosong (``null``). Sepeda motor tidak akan bisa dibuat kalau tidak ada wujud mesinnya.
3. **\*\*Boundary Check (Pembatasan Nilai) pada Behavior:\*\*** Pada method ``tambahKecepatan`` dan ``kurangiKecepatan``, kecepatan motor dicegah agar tidak melebihi ``maxSpeed``, dan dicegah agar tidak menjadi minus (di bawah 0 km/jam). Input angkanya pun dilarang negatif.

---

### ### 3. Pertimbangan Relasi Uses-A

**\*\*Apakah desain ini butuh relasi Uses-A?\*\***

Ya, sangat relevan jika kita ingin mensimulasikan interaksi dunia nyata tanpa harus mengikat objek tersebut selamanya.

\* Sebagai contoh, saya menambahkan kelas **\*\*`Mekanik`\*\***.

\* Method ``servisMotor(SepedaMotor motor)`` adalah bentuk **\*\*Uses-A (Dependency)\*\***.

\* Mekanik *\*menggunakan\** objek Sepeda Motor untuk diservis sebentar saja, tapi Mekanik tersebut tidak "memiliki" motor itu (bukan **\*Has-A\***). Dia hanya menerimanya melalui parameter fungsi, lalu selesai.

---

### ### 4. Diagram UML (Teks / Visualisasi)

Berikut adalah gambaran Diagram UML-nya.

\*Catatan: Kamu bisa menyalin kode "Mermaid" di bawah ini dan mem-paste nya di [Mermaid Live Editor](https://mermaid.live/) untuk melihat gambarnya secara visual.\*

```
```mermaid
```

```
classDiagram
```

```
class Mesin {  
    - int kapasitas  
    - String tipeBahanBakar  
    + getKapasitas() int  
    + setKapasitas(int)  
    + getTipeBahanBakar() String  
    + setTipeBahanBakar(String)  
}
```

```
class SepedaMotor {  
    - String merek  
    - String warna  
    - int maxSpeed  
    - int kecepatanSaatIni  
    - Mesin mesin  
    + tambahKecepatan(int)  
    + kurangiKecepatan(int)  
    + getMesin() Mesin  
    + setMesin(Mesin)  
}
```

```
class Mekanik {  
    - String namaMekanik  
    + servisMotor(SepedaMotor) void  
}
```

%% Relasi Has-A (Komposisi - Belah ketupat padat)

%% 1 Sepeda motor tepat memiliki 1 mesin

SepedaMotor "1" *-- "1" Mesin : Has-A (Composition)

%% Relasi Uses-A (Dependency - Panah putus-putus)

Mekanik ..> SepedaMotor : Uses-A

'''

****Penjelasan Multiplicity (Angka pada relasi):****

* "1" *-- "1" : Ini berarti ****1**** buah objek `SepedaMotor` pasti memiliki tepat ****1**** buah objek `Mesin`. Belah ketupat padat (`\*--`) menunjukkan ini adalah relasi yang sangat kuat (Composition). Jika motornya dihancurkan di pabrik, ya mesinnya ikutan hancur/menjadi bagian darinya.